Riverbend — Evaluation Results (40 sample messages)

Reproducible with one command. Two modes; **deterministic is authoritative for this submission**, the real-model path is documented and partially measured (see §Real-model).

- `make eval` → deterministic (recorded golden extraction → real code pipeline; free) → writes `docs/results.md`
- `make eval-live` → real OpenAI via the LiteLLM proxy (paid; measures tokens + LiteLLM-calculated cost)

Execution fidelity (honest scope). The harness exercises the real `intake → (gateway) → validate → decide → catalog-price → order` services and real Postgres, but drives the pipeline **in-process** — it does not replay the `pg-boss` queue/worker hop (that path has its own integration tests). Email-channel samples are injected as intake, not delivered through a real inbox; the new inbound-email path (D25) is covered by `tests/integration/email-new-inbound.test.ts`, not by a live Resend delivery here. No claim of live inbox delivery or live-model results is made from this deterministic run.

Run metadata (deterministic)

Field	Value
Evaluation date	2026-11-04
Dataset	`eval/messages.json` — the brief's 40 sample messages (3 shops, email+web), incl. the 8 tricky cases
Model / provider	**none** (deterministic) — golden expected extraction feeds the real validate→price→decide pipeline
Execution mode	`recorded` (deterministic)
Messages processed	40
Work orders created	**32**
Follow-ups requested	**4**
Failures (model/API)	**0**
Other valid business outcomes	answered (price): 1 · needs_review: 2 · duplicate (idempotent replay): 1
Total model calls	0 (no live calls)
Input / output tokens	0 / 0 (deterministic)
Total model cost	**$0.00 real** — reserve *estimate* only: 74 000 micros ≈ **$0.0740** (NOT real spend)

Field	Value
Avg cost / message	n/a (deterministic)
Total work-order value	**$1 865.00** (catalog-priced in code)
Run duration	~a few seconds (all ≤30 ms/message after DB warm-up)

Counting rules (explicit)

- **Work order created** = request outcome `ordered` (a priced `work_orders` row exists). → 32
- **Follow-up requested** = outcome `awaiting_customer` (exactly one clarification, I6). → 4
- **Failure** = a model/API error or invalid-output rejection only. Valid *business* outcomes are **not**

failures: `answered` (price inquiry, 1), `needs_review` (wrong-shop + cancellation, 2), `duplicate` (idempotent replay, 1). → failures = 0.

- 32 + 4 + 1 + 2 + 1 = **40** ✓ (consistent with the per-message rows).

Summary table

Total messages	Work orders created	Follow-ups	Failures	Total model spend
40	32	4	0	$0.00 real (est. $0.074; deterministic)

Per-message results (deterministic)

Expected outcome in `recorded` mode = the golden-defined outcome; **actual matched expected for all 40** (deterministic). Model cost is the per-call reserve estimate (2 000 micros) where the pipeline would call the model; `–` where no call happens (short-circuited business outcomes). Full table also in `docs/results.md`.

Message	Expected = Actual	WO created	Follow-up	Failure	Model cost (est.)
m-1001	ordered TUNE_BASIC×1 ($65)	✓			2000µ
m-1002	ordered FLAT_FIX×1 ($15)	✓			2000µ
m-1003	ordered GEAR_ADJUST×1 ($25)	✓			2000µ
m-1004	ordered BRAKE_PADS×1 ($35)	✓			2000µ

Message	Expected = Actual	WO created	Follow-up	Failure	Model cost (est.)
m-1005	ordered BRAKE_ADJUST+GEAR_ADJUST ($43)	✓			2000µ
m-1006	ordered WHEEL_TRUE×1 ($30)	✓			2000µ
m-1007	ordered TUNE_FULL×1 ($185)	✓			2000µ
m-1008	ordered EBIKE_DIAG×1 ($60)	✓			2000µ
m-1009	needs_review (wrong shop)			not a failure (business)	—
m-1010	ordered CHAIN_REPLACE×1 ($45)	✓			2000µ
m-1011	ordered BRAKE_BLEED×1 ($40)	✓			2000µ
m-1012	ordered GENERAL_INSPECT×1 ($25)	✓			2000µ
m-1013	awaiting_customer (ambiguous)		✓		2000µ
m-1003 (resend)	duplicate (idempotent replay)			not a failure	—
m-1015	ordered TUNE_BASIC×1 ($65)	✓			2000µ
m-1016	ordered GEAR_ADJUST×1 ($25)	✓			2000µ
m-1017	ordered BRAKE_ADJUST×2 ($36)	✓			2000µ
m-1018	answered (price inquiry)			not a failure (business)	2000µ
m-1019	ordered WHEEL_TRUE×1 ($30)	✓			2000µ
m-1020	awaiting_customer (ambiguous)		✓		2000µ
m-1021	awaiting_customer (ambiguous)		✓		2000µ
m-1022	ordered EBIKE_DIAG×1 ($60)	✓			2000µ
m-1023	ordered TUNE_BASIC+WHEEL_TRUE ($95)	✓			2000µ
m-1024	ordered CHAIN_REPLACE×1 ($45) — prompt-injection ignored	✓			2000µ
m-1025	ordered GENERAL_INSPECT×1 ($25) — "whatever it needs"	✓			2000µ
m-1026	ordered GEAR_ADJUST×1 ($25)	✓			2000µ
m-1027	awaiting_customer (ambiguous)		✓		2000µ

Message	Expected = Actual	WO created	Follow-up	Failure	Model cost (est.)
m-1028	ordered GEAR_ADJUST×1 ($25)	✓			2000µ
m-1029	ordered BRAKE_BLEED×2+BRAKE_PADS×2 ($150)	✓			2000µ
m-1030	ordered FLAT_FIX×1 ($15)	✓			2000µ
m-1031	ordered WHEEL_TRUE×2 ($60)	✓			2000µ
m-1032	ordered TUNE_BASIC×1 ($65) — single bike	✓			2000µ
m-1033	ordered BRAKE_ADJUST×2+GEAR_ADJUST ($61) — Spanish	✓			2000µ
m-1034	ordered CHAIN_REPLACE×1 ($45)	✓			2000µ
m-1035	ordered TIRE_REPLACE×2 ($40)	✓			2000µ
m-1036	ordered EBIKE_DIAG×1 ($60)	✓			2000µ
m-1037	ordered BRAKE_BLEED×1 ($40)	✓			2000µ
m-1038	needs_review (cancellation → staff, D23)			not a failure (business)	—
m-1039	ordered FLAT_FIX+TUNE_FULL ($200)	✓			2000µ
m-1040	ordered TUNE_BASIC×2 ($130) — two bikes	✓			2000µ

(Tricky-case notes — prompt injection, idempotent resend, wrong-shop, price inquiry, delegation, cancellation, Spanish, ambiguous — are in `docs/results.md §Tricky cases`.)

Current production model: gpt-5.4-mini (D26, upgraded 2026-11-06)

The backing model was upgraded `openai/gpt-4o-mini` → **`openai/gpt-5.4-mini-2026-03-17`** (alias `riverbend-extraction` unchanged; config-only rollback preserved). Full measured live run, per-tenant cost accounting (reconciled against LiteLLM's own team spend logs), and a side-by-side quality/cost comparison are in **`docs/results-live-gpt-5.4-mini.md`** and **`docs/MODEL_COST_REPORT.md`**.

gpt-5.4-mini live run (2026-11-06): 40 messages · 38 real calls · **33 ordered / 3 follow-ups / 2 reviews / 1 answered / 1 duplicate / 0 failures** (reconciles to 40) · 28 363 tokens (25 852/2 511) · **measured spend $0.0307** (30 692 micros; per-tenant sum = run total, exact) · $1 841 order value. Per shop: rb-downtown $0.0103, rb-lakeshore $0.0096, rb-northside $0.0107 — each on its own LiteLLM

team UUID (verified against `/spend/logs`). All deterministic guards held (wrong-shop, injection, delegation, bike-count, cancellation).

The gpt-4o-mini run below is **preserved unchanged as historical evidence** (and in `docs/results-live-gpt-4o-mini.md`).

Historical real-model run — gpt-4o-mini (2026-11-06, superseded by D26)

A real run executed 2026-11-06 against OpenAI through the LiteLLM proxy. Numbers are read back from the DB and from LiteLLM's own accounting — nothing here is estimated or synthesized.

Field	Value
Evaluation date	**2026-11-06**
Model ID (backing)	**`openai/gpt-4o-mini`** (superseded — see current model above)
Provider / proxy	OpenAI via LiteLLM proxy (per-shop virtual keys, `BUDGET_AUTHORITY=litellm`)
LiteLLM alias	**`riverbend-extraction`**
Messages processed	**40**
Real OpenAI calls	**38** (2 messages reach an outcome before any model call: m-1009 is routed by the shop-mismatch guard; #14 is an idempotent duplicate)
Work orders created (`ordered`)	**36**
Clarifications (`awaiting_customer`)	**0** (the live model extracted directly on the borderline messages the golden run flags as questions)
Answered (price inquiry)	**1** (m-1018)
Staff review (`needs_review`)	**2** (m-1009 wrong-shop, m-1038 cancellation → D23)
Duplicates (idempotent replay)	**1** (m-1003 resend, #14)
Failures (model/API error or invalid output)	**0**
Held (budget)	**0**
Total input / output tokens	**26 004 / 2 136** (28 140 total)
Actual model spend (LiteLLM `x-litellm-response-cost`)	**5 183 micros = $0.0052**
Independent corroboration	LiteLLM `/global/spend` → **$0.005185** (matches)

Field	Value
Total work-order value (catalog-priced in code)	**$2 014.00**
Spend guard / proxy aggregate cap	$1.00 hard stop / $15 proxy cap (assignment limit $20)

Reconciliation: 36 ordered + 1 answered + 2 needs_review + 1 duplicate = **40** ✓.

Tricky cases (actual live outcomes): prompt injection m-1024 → `ordered [CHAIN_REPLACE×1]` (injection ignored; code prices from catalog); resend m-1003 → `duplicate`; price inquiry m-1018 → `answered`; wrong-shop m-1009 → `needs_review` (guard, pre-model); delegation m-1025 → `ordered [GENERAL_INSPECT×1]` (guard); single-bike m-1032 → `ordered [TUNE_BASIC×1]` (bike-count guard); cancellation m-1038 → `needs_review` (D23). Full per-message table + model-vs-golden divergences in `docs/results-live.md`.

Classification divergences from the golden run (honest; model variance, not code defects). On ambiguous messages the deterministic golden run flags as one-question follow-ups, the live `gpt-4o-mini` chose to extract directly (e.g. m-1013, m-1020, m-1021, m-1027 → `ordered` instead of `awaiting_customer`), and on two messages it selected a different-but-plausible service/quantity than the golden (m-1019 `BRAKE_ADJUST` vs golden `WHEEL_TRUE`; m-1033 added `CHAIN_REPLACE`). These shift the ordered/follow-up split (live 36/0 vs deterministic 32/4) but **every code invariant held**: catalog-only pricing (I5), idempotency (m-1003), tenant isolation (I1), schema validation (I3), and the budget cap (I4). This is why the **deterministic** run remains authoritative for regression: it pins the model's interpretation so decisions/prices are reproducible.

Reproduction:

```
docker compose --profile llm up -d db litellm      # proxy holds OPENAI_API_KEY; isolated LiteLLM Postgres
EVAL_LLM_BASE_URL=http://localhost:4000/v1 make eval-live   # 40 messages, real OpenAI; writes docs/results-liv
```

The harness uses an isolated Testcontainer Postgres (no app/customer data touched) and sends no customer email. Historical runs preserved separately: `docs/results.md` (deterministic), `docs/results-live-before-guards.md` (pre-guard live). If no real key is configured, `make eval` (deterministic) is authoritative and real-model spend is **NOT MEASURED** — never synthesized.